Министерство образования Республики Беларусь
Учреждение образования
«Брестский государственный технический университет»
Кафедра ИИТ

Лабораторная работа №5
По дисциплине: “СПП”

Выполнил:
Студент 3 курса
Группы ПО-12
Мартынюк В. В.
Проверила:
Кулик А. Д.

Брест 2026

Цель работы: приобрести практические навыки разработки API и баз данных

10. Автоматический анализ самых активных контрибьюторов в репозитории

Условие:

Требования к оформлению отчета

Отчет по лабораторной работе должен содержать следующие разделы (примеры оформления отчетов можно найти в папке с заданиями):

1) Изложение цели работы.
2) Задание по лабораторной работе с описанием своего варианта.
3) Спецификации ввода-вывода программы.
4) Текст программы (кратко).
5) Выводы по проделанной работе.

Общее задание

1. Реализовать базу данных из не менее 5 таблиц на заданную тематику. При реализации продумать типизацию полей и внешние ключи в таблицах;
2. Визуализировать разработанную БД с помощью схемы, на которой отображены все таблицы и связи между ними (пример, схема на рис. 1);
3. На языке Python с использованием SQLAlchemy реализовать подключение к БД;
4. Реализовать основные операции с данными (выборку, добавление, удаление, модификацию);
5. Для каждой реализованной операции с использованием FastAPI реализовать отдельный эндпойнт;

10) База данных Учет успеваемости

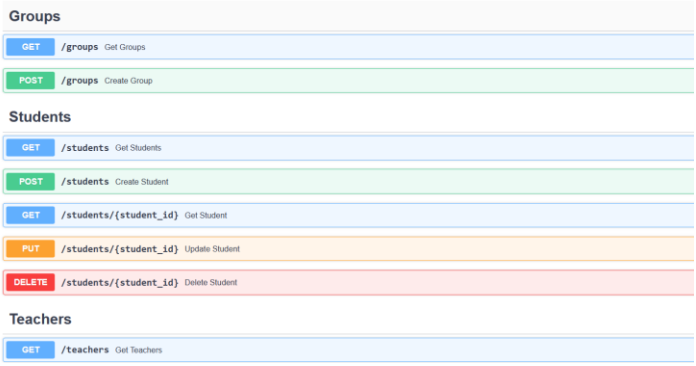
Решение:

```
from fastapi import FastAPI, Depends, HTTPException, status
from sqlalchemy.orm import Session
from typing import List

from database import engine, get_db
from models import Base
import crud
import schemas

Base.metadata.create_all(bind=engine)

app = FastAPI(
    title="API учёта успеваемости",
    description="Лабораторная работа №5 - Учёт успеваемости студентов",
    version="1.0.0"
)


@app.get("/groups", response_model=List[schemas.GroupResponse], tags=["Groups"])
def get_groups(db: Session = Depends(get_db)):
    return crud.get_groups(db)


@app.post("/groups",       response_model=schemas.GroupResponse,       status_code=status.HTTP_201_CREATED,
tags=["Groups"])
def create_group(group: schemas.GroupCreate, db: Session = Depends(get_db)):
    return crud.create_group(db, group)
```

```
@app.get("/students", response_model=List[schemas.StudentResponse], tags=["Students"])
def get_students(skip: int = 0, limit: int = 100, db: Session = Depends(get_db)):
    return crud.get_students(db, skip=skip, limit=limit)


@app.get("/students/{student_id}", response_model=schemas.StudentResponse, tags=["Students"])
def get_student(student_id: int, db: Session = Depends(get_db)):
    student = crud.get_student(db, student_id)
    if not student:
        raise HTTPException(status_code=404, detail="Студент не найден")
    return student


@app.post("/students",     response_model=schemas.StudentResponse,     status_code=status.HTTP_201_CREATED,
tags=["Students"])
def create_student(student: schemas.StudentCreate, db: Session = Depends(get_db)):
    group = crud.get_group(db, student.group_id)
    if not group:
        raise HTTPException(status_code=404, detail="Группа не найдена")
    return crud.create_student(db, student)


@app.put("/students/{student_id}", response_model=schemas.StudentResponse, tags=["Students"])
def update_student(student_id: int, student: schemas.StudentUpdate, db: Session = Depends(get_db)):
    updated = crud.update_student(db, student_id, student)
    if not updated:
        raise HTTPException(status_code=404, detail="Студент не найден")
    return updated


@app.delete("/students/{student_id}", tags=["Students"])
def delete_student(student_id: int, db: Session = Depends(get_db)):
    deleted = crud.delete_student(db, student_id)
    if not deleted:
        raise HTTPException(status_code=404, detail="Студент не найден")
    return {"message": f"Студент с ID {student_id} удалён", "deleted_id": student_id}


@app.get("/teachers", response_model=List[schemas.TeacherResponse], tags=["Teachers"])
def get_teachers(db: Session = Depends(get_db)):
    return crud.get_teachers(db)


@app.post("/teachers",     response_model=schemas.TeacherResponse,     status_code=status.HTTP_201_CREATED,
tags=["Teachers"])
def create_teacher(teacher: schemas.TeacherCreate, db: Session = Depends(get_db)):
    return crud.create_teacher(db, teacher)


@app.get("/subjects", response_model=List[schemas.SubjectResponse], tags=["Subjects"])
def get_subjects(db: Session = Depends(get_db)):
    return crud.get_subjects(db)


@app.post("/subjects",     response_model=schemas.SubjectResponse,     status_code=status.HTTP_201_CREATED,
tags=["Subjects"])
def create_subject(subject: schemas.SubjectCreate, db: Session = Depends(get_db)):
    teacher = crud.get_teacher(db, subject.teacher_id)
    if not teacher:
        raise HTTPException(status_code=404, detail="Преподаватель не найден")
    return crud.create_subject(db, subject)


@app.get("/grades/{student_id}", response_model=List[schemas.GradeResponse], tags=["Grades"])
def get_student_grades(student_id: int, db: Session = Depends(get_db)):
    student = crud.get_student(db, student_id)
    if not student:
        raise HTTPException(status_code=404, detail="Студент не найден")
    return crud.get_grades_by_student(db, student_id)


@app.post("/grades",       response_model=schemas.GradeResponse,       status_code=status.HTTP_201_CREATED,
tags=["Grades"])
def add_grade(grade: schemas.GradeCreate, db: Session = Depends(get_db)):
    student = crud.get_student(db, grade.student_id)
```

```
    if not student:
        raise HTTPException(status_code=404, detail="Студент не найден")

    subject = crud.get_subject(db, grade.subject_id)
    if not subject:
        raise HTTPException(status_code=404, detail="Предмет не найден")

    if grade.value < 0 or grade.value > 100:
        raise HTTPException(status_code=400, detail="Оценка должна быть от 0 до 100")

    return crud.create_grade(db, grade)


@app.get("/average/{student_id}", tags=["Grades"])
def get_average_grade(student_id: int, db: Session = Depends(get_db)):
    student = crud.get_student(db, student_id)
    if not student:
        raise HTTPException(status_code=404, detail="Студент не найден")

    avg = crud.get_average_grade(db, student_id)
    return {"student_id": student_id, "student_name": student.name, "average_grade": round(avg, 2)}


@app.get("/", tags=["Root"])
def root():
    return {
        "message": "API учёта успеваемости",
        "endpoints": {
            "groups": "/groups",
            "students": "/students",
            "teachers": "/teachers",
            "subjects": "/subjects",
            "grades": "/grades/{student_id}",
            "average": "/average/{student_id}"
        },
        "docs": "/docs"
    }
```

```
1. Создаём группу...
   Статус: 201
   Ответ: {'name': 'ИВТ-41', 'id': 1}

2. Создаём преподавателя...
   Статус: 201
   Ответ: {'name': 'Смирнова Анна Петровна', 'id': 1}

3. Создаём предмет...
   Статус: 201
   Ответ: {'name': 'Python', 'teacher_id': 1, 'id': 1}

4. Создаём студента...
   Статус: 201
   Ответ: {'name': 'Сидоров Алексей', 'group_id': 1, 'id': 1}
```



Вывод: В результате выполнения лабораторной работы были закреплены навыки разработки API и баз данных.